Three.js & Babylon.js

Build 3D Experiences for the Web

Two powerful JavaScript libraries for 3D graphics in the browser

Scenes, cameras, lights, materials, models, animation, interaction, and mini projects

3D

WebGL

Interactive

OVERVIEW

3D on the Web

Modern browsers can draw real-time 3D graphics using **WebGL** (and the newer **WebGPU**) with no plugins. Writing raw WebGL is long and difficult, so developers use libraries that handle the hard parts. The two most popular JavaScript choices are **Three.js** and **Babylon.js**. You can use them to build product viewers, games, data visualizations, virtual tours, portfolios, and AR/VR experiences.

	Three.js	Babylon.js
Type	Lightweight 3D library	Full 3D game and rendering engine
Created by	Ricardo Cabello (mrdoob), 2010	Microsoft team (David Catuhe), 2013
Philosophy	Flexible building blocks; add what you need	Batteries included; many features built in
Written in	JavaScript	TypeScript
License	MIT	Apache 2.0
Best for	Websites, creative coding, visualizations	Games, large scenes, tools, XR apps

Core ideas shared by both

- **Scene:** the 3D world that holds every object, light, and camera
- **Camera:** the viewpoint through which you see the scene
- **Mesh:** a visible object = **geometry** (shape) + **material** (surface look)
- **Lights:** without lights, most materials appear black
- **Render loop:** the scene is redrawn about 60 times per second to create motion
- **Coordinates:** X = right, Y = up, Z = depth. Three.js is right-handed; Babylon.js is left-handed by default

> **Prerequisite:** Basic JavaScript (variables, functions, arrow functions, classes) and a little HTML. Basic vector and angle math helps, and radians are used for rotation: 180 degrees = `Math.PI`.

1 Three.js: Getting Started

Scene, camera, renderer, and your first cube

Three things you always need

Object	Role
Scene	Container for all objects and lights
Camera	PerspectiveCamera(fov, aspect, near, far) is the most common
Renderer	WebGLRenderer draws the scene to a <canvas> every frame

Install with `npm install three` or use a CDN import map. Complete first scene:

```
<script type="importmap">
{ "imports": {
  "three": "https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.module.js",
  "three/addons/": "https://cdn.jsdelivr.net/npm/three@0.160.0/examples/jsm/"
}}
</script>
<script type="module">
import * as THREE from "three";

const scene = new THREE.Scene();
scene.background = new THREE.Color(0x0f172a);

const camera = new THREE.PerspectiveCamera(
  75, innerWidth / innerHeight, 0.1, 100);
camera.position.z = 3;

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement);

const cube = new THREE.Mesh(
  new THREE.BoxGeometry(1, 1, 1),
  new THREE.MeshStandardMaterial({ color: 0x38bdf8 }));
scene.add(cube);

scene.add(new THREE.AmbientLight(0xffffff, 0.5));
const sun = new THREE.DirectionalLight(0xffffff, 2);
sun.position.set(3, 4, 5);
scene.add(sun);

renderer.setAnimationLoop(() => {
  cube.rotation.x += 0.01;
  cube.rotation.y += 0.01;
  renderer.render(scene, camera);
});
</script>
```

Geometries, materials, and lights

Geometries	Materials	Lights
BoxGeometry	MeshBasicMaterial (no light needed)	AmbientLight (soft overall light)
SphereGeometry	MeshStandardMaterial (realistic, PBR)	DirectionalLight (like the sun)
PlaneGeometry	MeshPhysicalMaterial (glass, clearcoat)	PointLight (light bulb)
CylinderGeometry	MeshPhongMaterial (shiny, older style)	SpotLight (cone of light)
TorusGeometry	MeshNormalMaterial (debugging colors)	HemisphereLight (sky and ground)
BufferGeometry (custom)	ShaderMaterial (custom GLSL)	RectAreaLight (soft panels)

Material properties and textures

```
const loader = new THREE.TextureLoader();
const material = new THREE.MeshStandardMaterial({
  map: loader.load("wood.jpg"),      // color texture
  roughness: 0.6,                    // 0 = shiny, 1 = matte
  metalness: 0.1,                    // 0 = plastic, 1 = metal
});
```

Transform: position, rotation, scale

```
mesh.position.set(1, 0.5, -2);
mesh.rotation.y = Math.PI / 4;       // radians
mesh.scale.set(2, 2, 2);
group.add(mesh1, mesh2);             // Group moves its children together
```

Shadows (4 steps)

```
renderer.shadowMap.enabled = true;       // 1. enable in renderer
sun.castShadow = true;                   // 2. light casts shadows
cube.castShadow = true;                  // 3. objects cast
floor.receiveShadow = true;              // 4. surfaces receive
```

> **Tip:** Use `renderer.setPixelRatio(Math.min(devicePixelRatio, 2))` to keep graphics sharp on phones without hurting performance.

Animation, controls, models, and picking

Time-based animation keeps speed the same on every screen. **OrbitControls** lets users rotate, zoom, and pan:

```
import { OrbitControls } from "three/addons/controls/OrbitControls.js";

const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;

const clock = new THREE.Clock();
renderer.setAnimationLoop(() => {
  const t = clock.getElapsedTime();
  cube.position.y = Math.sin(t) * 0.5;     // smooth up and down
  controls.update();
  renderer.render(scene, camera);
});
```

Loading 3D models (GLTF / GLB format)

```
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";

let mixer;
new GLTFLoader().load("robot.glb", (gltf) => {
  scene.add(gltf.scene);
  mixer = new THREE.AnimationMixer(gltf.scene);       // play built-in animation
  mixer.clipAction(gltf.animations[0]).play();
});
// in the loop:  mixer?.update(clock.getDelta());
```

Clicking objects with a Raycaster

```
const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();

addEventListener("click", (e) => {
  pointer.x =  (e.clientX / innerWidth)  * 2 - 1;
  pointer.y = -(e.clientY / innerHeight) * 2 + 1;
  raycaster.setFromCamera(pointer, camera);
  const hits = raycaster.intersectObjects(scene.children);
  if (hits.length) hits[0].object.material.color.set(0xff5555);
});
```

Handle window resizing

```
addEventListener("resize", () => {
  camera.aspect = innerWidth / innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(innerWidth, innerHeight);
});
```

2 Babylon.js: Getting Started

Engine, scene, camera, and your first box

Four things you always need

Object	Role
Engine	Connects to the <canvas> and runs the render loop
Scene	Container for meshes, lights, cameras, and materials
Camera	ArcRotateCamera orbits a target; FreeCamera / UniversalCamera for walking
Light	HemisphericLight, DirectionalLight, PointLight, SpotLight

Install with `npm install @babylonjs/core` or load from the CDN. Complete first scene:

```
<canvas id="renderCanvas" style="width:100%; height:100vh"></canvas>
<script src="https://cdn.babylonjs.com/babylon.js"></script>
<script>
const canvas = document.getElementById("renderCanvas");
const engine = new BABYLON.Engine(canvas, true);
const scene  = new BABYLON.Scene(engine);

const camera = new BABYLON.ArcRotateCamera("cam",
  -Math.PI / 2, Math.PI / 2.5, 5, BABYLON.Vector3.Zero(), scene);
camera.attachControl(canvas, true);       // mouse / touch control

new BABYLON.HemisphericLight("light",
  new BABYLON.Vector3(0, 1, 0), scene);

const box = BABYLON.MeshBuilder.CreateBox("box", { size: 1 }, scene);
const mat = new BABYLON.StandardMaterial("mat", scene);
mat.diffuseColor = new BABYLON.Color3(0.2, 0.6, 0.9);
box.material = mat;

scene.onBeforeRenderObservable.add(() => {
  box.rotation.y += 0.01;
});

engine.runRenderLoop(() => scene.render());
window.addEventListener("resize", () => engine.resize());
</script>
```

> **Tip:** Babylon.js has a free online editor, the **Playground** (playground.babylonjs.com), where you can write and share code instantly without installing anything.

Meshes, materials, and lights

MeshBuilder shapes	Materials	Cameras
CreateBox	StandardMaterial (simple, fast)	ArcRotateCamera (orbit around target)
CreateSphere	PBRMaterial (realistic metal and rough)	UniversalCamera (keyboard + mouse walk)
CreateGround	NodeMaterial (visual shader editor)	FollowCamera (chases a mesh)
CreateCylinder	GridMaterial, WaterMaterial (extras)	FreeCamera (flying)
CreateTorus	MultiMaterial (several on one mesh)	XR cameras (VR / AR)
CreatePlane, CreateText	ShaderMaterial (custom GLSL)	

Materials and textures

```
const pbr = new BABYLON.PBRMaterial("metal", scene);
pbr.albedoColor = new BABYLON.Color3(0.9, 0.8, 0.3);
pbr.metallic  = 1;                  // 0 = plastic, 1 = metal
pbr.roughness = 0.3;                // 0 = mirror, 1 = matte

const wood = new BABYLON.StandardMaterial("wood", scene);
wood.diffuseTexture = new BABYLON.Texture("wood.jpg", scene);
```

Transform and parenting

```
sphere.position = new BABYLON.Vector3(1, 0.5, -2);
sphere.rotation.y = Math.PI / 4;          // radians
sphere.scaling = new BABYLON.Vector3(2, 2, 2);
sphere.parent = box;                      // moves with its parent
```

Shadows

```
const sun = new BABYLON.DirectionalLight("sun",
  new BABYLON.Vector3(-1, -2, -1), scene);
const shadows = new BABYLON.ShadowGenerator(1024, sun);
shadows.addShadowCaster(sphere);
ground.receiveShadows = true;
```

Animation, interaction, models, UI, and physics

Built-in animation helper and **click picking**:

```
BABYLON.Animation.CreateAndStartAnimation(
  "slide", box, "position.x", 30, 60, -2, 2,      // fps, frames, from, to
  BABYLON.Animation.ANIMATIONLOOPMODE_CYCLE);

scene.onPointerObservable.add((info) => {
  if (info.type === BABYLON.PointerEventTypes.POINTERDOWN
      && info.pickInfo.hit) {
    console.log("Clicked:", info.pickInfo.pickedMesh.name);
  }
});
```

Loading 3D models (needs the loaders script)

```
<script src="https://cdn.babylonjs.com/loaders/babylonjs.loaders.min.js"></script>

const result = await BABYLON.SceneLoader.ImportMeshAsync(
  "", "/models/", "robot.glb", scene);
result.meshes[0].scaling.scaleInPlace(0.5);
result.animationGroups[0]?.start(true);       // loop built-in animation
```

2D user interface (needs babylon.gui.min.js)

```
const ui  = BABYLON.GUI.AdvancedDynamicTexture.CreateFullscreenUI("UI");
const btn = BABYLON.GUI.Button.CreateSimpleButton("btn", "Change color");
btn.width = "160px";  btn.height = "40px";
btn.color = "white";  btn.background = "#BB464B";
btn.onPointerUpObservable.add(() => {
  mat.diffuseColor = BABYLON.Color3.Random();
});
ui.addControl(btn);
```

Powerful built-in extras

- **Inspector:** `scene.debugLayer.show()` opens a live debugger for the whole scene
- **Physics:** Havok engine support for gravity, collisions, and forces
- **WebXR:** `scene.createDefaultXRExperienceAsync()` for VR and AR
- **Particles, post-processing, sound, and a Node Material Editor**

SIDE BY SIDE

Same Task, Two Libraries

Here is how the key steps compare when drawing a spinning blue sphere:

Step	Three.js	Babylon.js
Create scene	new THREE.Scene()	new BABYLON.Scene(engine)
Renderer / engine	new THREE.WebGLRenderer()	new BABYLON.Engine(canvas, true)
Camera	new THREE.PerspectiveCamera(...)	new BABYLON.ArcRotateCamera(...)
Create a sphere	new THREE.Mesh(geometry, material)	BABYLON.MeshBuilder.CreateSphere(...)
Add to scene	scene.add(mesh)	Added automatically when created
Rotate	mesh.rotation.y += 0.01	mesh.rotation.y += 0.01
Render loop	renderer.setAnimationLoop(fn)	engine.runRenderLoop(fn)
Orbit controls	import OrbitControls (addon)	camera.attachControl(canvas)
Click detection	Raycaster	scene.pick() / pointer observable

Which should you choose?

Choose Three.js when...	Choose Babylon.js when...
You want maximum flexibility and a huge community of examples	You want many tools built in: inspector, physics, GUI, XR
You are building a creative website or visual effect	You are building a game or large interactive app
You plan to use React Three Fiber (React + Three.js)	You prefer TypeScript and a full engine structure
Small bundle size and custom shaders matter most	You want strong tooling like Playground and Node Material Editor

Honest answer: Both are excellent. Learn the shared concepts (scene, camera, mesh, light, render loop) once, and switching between them becomes easy. Pick the one whose style you enjoy more and build something.

PUTTING IT ALL TOGETHER

Mini Projects

Project A (Three.js): Orbiting planet. Uses spheres, lights, a Group, and time-based animation. Add this inside your first-scene setup in place of the cube:

```
const sunMesh = new THREE.Mesh(
  new THREE.SphereGeometry(1, 32, 32),
  new THREE.MeshBasicMaterial({ color: 0xffcc33 }));
scene.add(sunMesh);

const pivot = new THREE.Group();            // rotates around the sun
const earth = new THREE.Mesh(
  new THREE.SphereGeometry(0.3, 32, 32),
  new THREE.MeshStandardMaterial({ color: 0x3b82f6 }));
earth.position.x = 3;
pivot.add(earth);
scene.add(pivot, new THREE.PointLight(0xffffff, 80));

const clock = new THREE.Clock();
renderer.setAnimationLoop(() => {
  const dt = clock.getDelta();
  pivot.rotation.y += dt * 0.8;             // orbit
  earth.rotation.y += dt * 3;               // spin
  renderer.render(scene, camera);
});
```

Project B (Babylon.js): Click-to-recolor bouncing ball. Uses ground, a sphere, animation, and picking:

```
const ground = BABYLON.MeshBuilder.CreateGround("g",
  { width: 6, height: 6 }, scene);
const ball = BABYLON.MeshBuilder.CreateSphere("ball",
  { diameter: 1 }, scene);
const bm = new BABYLON.StandardMaterial("bm", scene);
bm.diffuseColor = BABYLON.Color3.Red();
ball.material = bm;

BABYLON.Animation.CreateAndStartAnimation("bounce", ball, "position.y",
  30, 60, 0.5, 2.5, BABYLON.Animation.ANIMATIONLOOPMODE_YOYO);

scene.onPointerObservable.add((info) => {
  if (info.type === BABYLON.PointerEventTypes.POINTERDOWN
      && info.pickInfo.pickedMesh === ball) {
    bm.diffuseColor = BABYLON.Color3.Random();
  }
});
```

NEXT STEPS

Learning Roadmap & Cheat Sheet

Stage	Focus	Practice project
1. Core concepts	Scene, camera, renderer/engine, meshes, lights	Spinning cube or box
2. Materials	Colors, textures, PBR, shadows	Textured planet or room
3. Interaction	Orbit controls, raycasting / picking, resize	Clickable product colors
4. Models	GLTF/GLB loading, model animations	3D product or character viewer
5. Advanced	Custom shaders, post-processing, performance	Visual effect landing page
6. Physics and UI	Havok / Rapier physics, HTML or GUI overlays	Simple 3D game
7. Frameworks	React Three Fiber, or Babylon.js with TypeScript	Full 3D portfolio website

Quick cheat sheet

Task	Three.js	Babylon.js
Install	npm i three	npm i @babylonjs/core
Box	new THREE.BoxGeometry(1,1,1)	MeshBuilder.CreateBox('b',{size:1})
Color	new THREE.Color(0xff0000)	new BABYLON.Color3(1,0,0)
Vector	new THREE.Vector3(x,y,z)	new BABYLON.Vector3(x,y,z)
Resize	renderer.setSize(w,h)	engine.resize()
Debug	Stats.js, lil-gui	scene.debugLayer.show()
Best docs	threejs.org/docs and /examples	doc.babylonjs.com and Playground

Best practices

- Use **GLB** models, compress textures, and keep polygon counts low for smooth mobile performance
- Reuse geometries and materials instead of creating new ones for each object
- Always handle window resizing and limit pixel ratio
- Use lights sparingly; every extra light and shadow costs speed
- Free models: **Sketchfab**, **Poly Haven**, and **Kenney.nl** (check each license)

> **Remember:** Start tiny. A single cube that you control with the mouse teaches more than copying a huge example you do not understand.